Aston University
Computational Intelligence

Implementing Particle Swarm Optimization

Expected Outcomes:
In this lab, you will be implementing two versions of Particle Swarm Optimization (PSO) for the Antenna Array Design Problem: one close to that discussed in lectures, and one of your own design.

IMPORTANT: A note on working together. You are welcome to work together with friends on the majority of this lab. However, the work for step 6 (creating a novel PSO variant) may be used as part of your assessment, so it is important that, for step 6, you create and implement the part of your own method which is different from the algorithm you were asked to implement in steps 1-5.

Instructions:
Recall the antenna array design problem.

Place a fixed number of antennae on an array such that the peak Side Lobe Level (SSL) is minimised, subject to the constraints that (1) the aperture size is exactly equal to half the number of antennae and that (2) no antenna is within a distance of 0.25 of another.

For this week's practical work, we are going to implement Particle Swarm Optimization (PSO) and apply it to the antenna array design problem.

Step 1: Creating a Problem Instance
Download the file `Antenna Design.zip` from Blackboard and extract its contents. It contains an implementation of an `AntennaArray` class which can be used to evaluate your solutions to the antenna array design problem.

Create an instance of the `AntennaArray` class. Its constructor takes two parameters: the first is the number of antennae to be placed and the second is the ``steering angle'' - the direction of the antenna array's main beam in degrees. For the moment, create an instance with three antennae and a steering angle of 90 degrees.

The class's `evaluate` method evaluates a design and returns the peak SSL. Use the method to evaluate a design in which the three antennae are regularly spaced along the aperture (at 0.5, 1.0 and 1.5): this should result in a peak SSL of 5.49. Use the ``Visualising the Problem'' slides from the Swarm Intelligence lecture, which show a graph of this problem, to see if you can find a design with lower peak SSL. Remember that the graph only shows the position of two of the antennae: the position of the third is fully determined by the constraint on aperture size.

Step 2: Generating Valid Solutions
Before we can attempt to optimize a design, we need to have a less labour-intensive way of generating solutions than doing so by hand! Write a method which takes a problem instance as input and, when called, randomly generates a valid design for that instance. As an initial attempt, I recommend the simple strategy discussed in the Antenna Array Design with PSO slides from the Swarm Intelligence lecture.

The `AntennaArray` class has some methods which you may find useful when doing this. The method `is_valid` takes a design as input and returns whether or not the design is valid. The `bounds` method returns an array containing lower and upper bounds on the position of each antenna. These bounds are not as tight as they could be - they only specify that every antenna must have a position between 0 and the desired aperture size. To have a reasonable chance of generating a valid solution we will need to make them tighter by factoring in some of the constraints. Again, see the Antenna Array Design with PSO slides from the Swarm Intelligence lecture for a starting point.

Once you have a method which generates random solutions, you can test how well you can solve the problem using random search. Generate and evaluate a series of antenna designs, keeping only the best one. What is the lowest peak SSL that you achieve using this method?

Step 3: Data Structure for a Particle
The pseudocode for a simple variant of Particle Swarm Optimization is:

```
BEGIN
  INITIALISE population;
  REPEAT UNTIL ( termination condition IS satisfied ) DO
    UPDATE global best;
    FOR EACH ( particle in population ) DO
      1. UPDATE velocity and position;
      2. EVALUATE new position;
      3. UPDATE personal best;
    OD
  OD
END
```

Most of the work involved in implementing PSO is in the implementation of the particles. Particles need to keep track of:

- their current *position*,
- their current *velocity*,
- their *personal best position*.

They should also keep track of the cost of their personal best position, so that they know when to update it. Drawing on your experience from step 2, you might also want to think about choosing a good solution representation.

Implement a particle which can keep track of these variables and, given a problem instance and the swarm's *global best position*, can update them. Make sure that you implement some **constraint handling** strategy for when the particle attempts to move to some invalid solution. The invisible wall strategy discussed in lectures will be sufficient.

Step 4: The Particle Swarm Optimization Algorithm
Once you have a working implementation of a particle, implement the swarm. A swarm is essentially a collection of particles. You should be able to initialise the swarm with appropriate randomly generated values (your method from task 2 should make this simple) and iterate the swarm (an iteration is an update of all particles). As you can see from the above pseudocode, the PSO algorithm simply consists of repeated swarm iterations.

Congratulations! You have now implemented PSO.

Step 5: Experimentation
Test the performance of PSO on the antenna design problem. How does it compare to random search for the placement of 3 antennae? What about 10? Similarly to when looking at evolutionary algorithms, it may be useful to monitor the algorithm's progress so you can better understand it. I would suggest displaying the lowest cost achieved at the end of each iteration.

Step 6: Create A Novel Variant of Particle Swarm Optimization
You have now implemented the PSO variant discussed in lectures. Can you improve on it? Keeping the termination condition the same, try and implement a different version of PSO. The velocity update equation of PSO defines its search strategy, so modifying some or all of it would be a good place to start.

You are also encouraged to consult the literature for inspiration when designing your novel variants. Many PSO variants are discussed in:

- Shami, T.M., El-Saleh, A.A., Alswaitti, M., Al-Tashi, Q., Summakieh, M.A. and Mirjalili, S., 2022. *Particle swarm optimization: A comprehensive survey*. IEEE Access, 10, pp.10031-10061.

You may wish to use the information in this paper to choose a variant and then to follow the reference in the paper to their original sources for full details.

Test your own variant of PSO against the one recommended above. Is yours better or worse? How would you decide? How confident are you in your answer??